

1. Is Python case sensitive when dealing with identifiers?

- a) yes
- b) no
- c) machine dependent
- d) none of the mentioned

Explanation: Case is always significant.

2. What is the maximum possible length of an identifier?

- a) 31 characters
- b) 63 characters
- c) 79 characters
- d) none of the mentioned

Explanation: Identifiers can be of any length.

3. Which of the following is invalid?

- a) `_a = 1`
- b) `__a = 1`
- c) `__str__ = 1`
- d) none of the mentioned

Explanation: All the statements will execute successfully but at the cost of reduced readability.

4. Which of the following is an invalid variable?

- a) `my_string_1`
- b) `1st_string`
- c) `foo`
- d) `_`

Explanation: Variable names should not start with a number.

5. Why are local variable names beginning with an underscore discouraged?

- a) they are used to indicate a private variables of a class
- b) they confuse the interpreter
- c) they are used to indicate global variables
- d) they slow down execution

Explanation: As Python has no concept of private variables, leading underscores are used to indicate variables that must not be accessed from outside the class.

6. Which of the following is not a keyword?

- a) `eval`
- b) `assert`
- c) `nonlocal`
- d) `pass`

Explanation: `eval` can be used as a variable.

7. All keywords in Python are in

- a) lower case
- b) UPPER CASE
- c) Capitalized
- d) None of the mentioned

Explanation: `True`, `False` and `None` are capitalized while the others are in lower case.

8. Which of the following is true for variable names in Python?

- a) unlimited length
- b) all private members must have leading and trailing underscores
- c) underscore and ampersand are the only two special characters allowed
- d) none of the mentioned

Explanation: Variable names can be of any length.

9. Which of the following is an invalid statement?

- a) `abc = 1,000,000`
- b) `a b c = 1000 2000 3000`**
- c) `a,b,c = 1000, 2000, 3000`
- d) `a_b_c = 1,000,000`

Explanation: Spaces are not allowed in variable names.

10. Which of the following cannot be a variable?

- a) `__init__`
- b) `in`**
- c) `it`
- d) `on`

Explanation: `in` is a keyword

11. Which of these is not a core datatype?

- a) Lists
- b) Dictionary
- c) Tuples
- d) Class**

Explanation: Class is a user defined datatype.

12. Given a function that does not return any value, What value is thrown by default when executed in shell.

- a) `int`
- b) `bool`
- c) `void`
- d) `None`**

Explanation: Python shell throws a `NoneType` object back.

13. Following set of commands are executed in shell, what will be the output?

```
>>>str="hello"  
>>>str[:2]  
>>>
```

- a) `he`**
- b) `lo`
- c) `olleh`
- d) `hello`

Explanation: We are printing only the 1st two bytes of string and hence the answer is “he”.

14. Which of the following will run without errors ?

- a) `round(45.8)`**
- b) `round(6352.898,2,5)`
- c) `round()`
- d) `round(7463.123,2,1)`

Explanation: Execute `help(round)` in the shell to get details of the parameters that are passed into the `round` function.

15. What is the return type of function `id` ?

- a) `int`**
- b) `float`
- c) `bool`
- d) `dict`

Explanation: Execute `help(id)` to find out details in python shell.`id` returns a integer value that is unique.

16. In python we do not specify types, it is directly interpreted by the compiler, so consider the following operation to be performed.

```
>>>x = 13 ? 2
```

objective is to make sure x has a integer value, select all that apply (python 3.xx)

- a) `x = 13 // 2`
- b) `x = int(13 / 2)`
- c) `x = 13 % 2`

d) All of the mentioned

Explanation: `//` is integer operation in python 3.0 and `int(..)` is a type cast operator.

17. What error occurs when you execute?

```
apple = mango
```

- a) `SyntaxError`
- b) `NameError`**
- c) `ValueError`
- d) `TypeError`

Explanation: Mango is not defined hence name error.

18. Carefully observe the code and give the answer.

```
def example(a):
```

```
    a = a + '2'
```

```
    a = a*2
```

```
    return a
```

```
>>>example("hello")
```

- a) indentation Error**
- b) cannot perform mathematical operation on strings
- c) hello2
- d) hello2hello2

Explanation: Python codes have to be indented properly.

19. What datatype is the object below ?

```
L = [1, 23, 'hello', 1].
```

- a) list**
- b) dictionary
- c) array
- d) tuple

Explanation: List datatype can store any values within it.

20. In order to store values in terms of key and value we use what core datatype.

- a) list
- b) tuple
- c) class
- d) dictionary**

Explanation: Dictionary stores values in terms of keys and values.

21. Which of the following results in a `SyntaxError` ?

- a) `""Once upon a time..."`, she said.'
- b) `"He said, 'Yes!'"`
- c) `'3\'`**
- d) `"""That's okay"""`

Explanation: Carefully look at the colons.

22. The following is displayed by a print function call:

1. tom
2. dick
3. harry

Select all of the function calls that result in this output

- a) `print("""tom
\ndick
\nharry""')`
- b) `print("""tomdickharry""')`
- c) `print('tom\n dick\n harry')`
- d) `print('tom
dick
harry')`

Explanation: The `\n` adds a new line.

23. What is the average value of the code that is executed below ?

```
>>>grade1 = 80
>>>grade2 = 90
>>>average = (grade1 + grade2) / 2
```

- a) 85
- b) 85.1
- c) 95
- d) 95.1

Explanation: Cause a decimal value to appear as output.

24. Select all options that print

hello-how-are-you

- a) `print('hello', 'how', 'are', 'you')`
- b) `print('hello', 'how', 'are', 'you' + '-' * 4)`
- c) `print('hello-' + 'how-are-you')`
- d) `print('hello' + '-' + 'how' + '-' + 'are' + 'you')`

Explanation: Execute in the shell.

25. What is the return value of `trunc()` ?

- a) int
- b) bool
- c) float
- d) None

Explanation: Execute `help(math.trunc)` to get details.

26. Which is the correct operator for power(x^y)?

- a) X^y
- b) $X^{**}y$
- c) $X^{^^}y$
- d) None of the mentioned

Explanation: In python, power operator is `x**y` i.e. `2**3=8`.

27. Which one of these is floor division?

- a) /
- b) //
- c) %
- d) None of the mentioned

Explanation: When both of the operands are integer then python chops out the fraction part and gives you the round off value, to get the accurate answer use floor division. This is floor division. For ex, $5/2 = 2.5$ but both of the operands are integer so answer of this expression in python is 2. To get the 2.5 answer, use floor division.

28. What is the order of precedence in python?

i) Parentheses

ii) Exponential

iii) Division

iv) Multiplication

v) Addition

vi) Subtraction

a) i,ii,iii,iv,v,vi

b) ii,i,iii,iv,v,vi

c) ii,i,iv,iii,v,vi

d) i,ii,iii,iv,vi,v

Explanation: For order of precedence, just remember this PEDMAS (similar to BODMAS)

29. What is answer of this expression, $22 \% 3$ is?

a) 7

b) 1

c) 0

d) 5

Explanation: Modulus operator gives remainder. So, $22\%3$ gives the remainder, that is, 1.

30. Mathematical operations can be performed on a string. State whether true or false.

a) True

b) False

Explanation: You can't perform mathematical operation on string even if the string is in the form: '1234...'.

31. Operators with the same precedence are evaluated in which manner?

a) Left to Right

b) Right to Left

c) Cant say

d) None of the mentioned

Explanation: None.

32. What is the output of this expression, $3*13$?**

a) 27

b) 9

c) 3

d) 1

Explanation: First this expression will solve $1**3$ because exponential have higher precedence than multiplication, so $1**3 = 1$ and $3*1 = 3$. Final answer is 3.

33. Which one of the following have the same precedence?

a) Addition and Subtraction

b) Multiplication and Division

c) Both a and b

d) None of the mentioned

Explanation: None.

34. The expression `Int(x)` implies that the variable x is converted to integer. State whether true or false.

a) True

b) False

Explanation: None.

35. Which one of the following have the highest precedence in the expression?

- a) Exponential
- b) Addition
- c) Multiplication
- d) Parentheses**

Explanation: Just remember: PEDMAS, that is, Parenthesis, Exponentiation, Division, Multiplication, Addition, Subtraction. Note that the precedence order of Division and Multiplication is the same. Likewise, the order of Addition and Subtraction is also the same.

36. What is the output of print 0.1 + 0.2 == 0.3?

- a) True
- b) False**
- c) Machine dependent
- d) Error

Explanation: Neither of 0.1, 0.2 and 0.3 can be represented accurately in binary. The round off errors from 0.1 and 0.2 accumulate and hence there is a difference of $5.5511e-17$ between $(0.1 + 0.2)$ and 0.3.

37. Which of the following is not a complex number?

- a) $k = 2 + 3j$
- b) $k = \text{complex}(2, 3)$
- c) $k = 2 + 3l$**
- d) $k = 2 + 3J$

Explanation: l (or L) stands for long.

38. What does ~4 evaluate to?

- a) -5**
- b) -4
- c) -3
- d) +3

Explanation: $\sim x$ is equivalent to $-(x+1)$.

39. What does ~~~~~5 evaluate to?

- a) +5**
- b) -11
- c) +11
- d) -5

Explanation: $\sim x$ is equivalent to $-(x+1)$.

40. Which of the following is incorrect?

- a) $x = 0b101$
- b) $x = 0x4f5$
- c) $x = 19023$
- d) $x = 03964$**

Explanation: Numbers starting with a 0 are octal numbers but 9 isn't allowed in octal numbers.

41. What is the result of cmp(3, 1)?

- a) 1**
- b) 0
- c) True
- d) False

Explanation: $\text{cmp}(x, y)$ returns 1 if $x > y$, 0 if $x == y$ and -1 if $x < y$.

42. Which of the following is incorrect?

- a) $\text{float}('inf')$
- b) $\text{float}('nan')$
- c) $\text{float}('56+'78')$

d) float('12+34')

Explanation: '+' cannot be converted to a float.

43. What is the result of round(0.5) – round(-0.5)?

a) 1.0

b) 2.0

c) 0.0

d) None of the mentioned

Explanation: Python rounds off numbers away from 0 when the number to be rounded off is exactly halfway through. round(0.5) is 1 and round(-0.5) is -1.

44. What does 3 ^ 4 evaluate to?

a) 81

b) 12

c) 0.75

d) 7

Explanation: ^ is the Binary XOR operator

45. Which module in Python supports regular expressions?

a) re

b) regex

c) pyregex

d) none of the mentioned

Explanation: re is a part of the standard library and can be imported using: import re.

46. What does the function re.match do?

a) matches a pattern at the start of the string

b) matches a pattern at any position in the string

c) such a function does not exist

d) none of the mentioned

Explanation: It will look for the pattern at the beginning and return None if it isn't found.

47. What does the function re.search do?

a) matches a pattern at the start of the string

b) matches a pattern at any position in the string

c) such a function does not exist

d) none of the mentioned

Explanation: It will look for the pattern at any position in the string.

48. What is the output of the following?

```
sentence = 'we are humans'
matched = re.match(r'(.*) (.*) (.*)', sentence)
print(matched.groups())
```

a) ('we', 'are', 'humans')

b) (we, are, humans)

c) ('we', 'humans')

d) 'we are humans'

Explanation: This function returns all the subgroups that have been matched.

49. What is the output of the following?

```
sentence = 'we are humans'
matched = re.match(r'(.*) (.*) (.*)', sentence)
print(matched.group())
```

a) ('we', 'are', 'humans')

b) (we, are, humans)

c) ('we', 'humans')

d) 'we are humans'

Explanation: This function returns the entire match.

50. What is the output of the following?

```
sentence = 'we are humans'
matched = re.match(r'(.*) (.*) (.*)', sentence)
print(matched.group(2))
```

- a) 'are'
- b) 'we'
- c) 'humans'
- d) 'we are humans'

Explanation: This function returns the particular subgroup.

51. What is the output of the following?

```
sentence = 'horses are fast'
regex = re.compile('(P<animal>\w+) (P<verb>\w+) (P<adjective>\w+)')
matched = re.search(regex, sentence)
print(matched.groupdict())
```

- a) {'animal': 'horses', 'verb': 'are', 'adjective': 'fast'}
- b) ('horses', 'are', 'fast')
- c) 'horses are fast'
- d) 'are'

Explanation: This function returns a dictionary that contains all the matches.

52. What is the output of the following?

```
sentence = 'horses are fast'
regex = re.compile('(P<animal>\w+) (P<verb>\w+) (P<adjective>\w+)')
matched = re.search(regex, sentence)
print(matched.groups())
```

- a) {'animal': 'horses', 'verb': 'are', 'adjective': 'fast'}
- b) ('horses', 'are', 'fast')
- c) 'horses are fast'
- d) 'are'

Explanation: This function returns all the subgroups that have been matched.

53. What is the output of the following?

```
sentence = 'horses are fast'
regex = re.compile('(P<animal>\w+) (P<verb>\w+) (P<adjective>\w+)')
matched = re.search(regex, sentence)
print(matched.group(2))
```

- a) {'animal': 'horses', 'verb': 'are', 'adjective': 'fast'}
- b) ('horses', 'are', 'fast')
- c) 'horses are fast'
- d) 'are'

Explanation: This function returns the particular subgroup

54. What is the output of the following?

```
x = ['ab', 'cd']
for i in x:
    i.upper()
print(x)
```

- a) ['ab', 'cd'].
- b) ['AB', 'CD'].
- c) [None, None].
- d) none of the mentioned

Explanation: The function upper() does not modify a string in place, it returns a new string which isn't being stored anywhere.

55. What is the output of the following?

```
x = ['ab', 'cd']
for i in x:
    x.append(i.upper())
print(x)
```

- a) ['AB', 'CD'].
- b) ['ab', 'cd', 'AB', 'CD'].
- c) ['ab', 'cd'].
- d) none of the mentioned**

Explanation: The loop does not terminate as new elements are being added to the list in each iteration.

56. What is the output of the following?

```
i = 1
while True:
    if i%3 == 0:
        break
    print(i)
    i + = 1
```

- a) 1 2
- b) 1 2 3
- c) error**
- d) none of the mentioned

Explanation: SyntaxError, there shouldn't be a space between + and = in +=.

57. What is the output of the following?

```
i = 1
while True:
    if i%0O7 == 0:
        break
    print(i)
    i += 1
```

- a) 1 2 3 4 5 6**
- b) 1 2 3 4 5 6 7
- c) error
- d) none of the mentioned

Explanation: Control exits the loop when i becomes 7.

58. What is the output of the following?

```
i = 5
while True:
    if i%0O11 == 0:
        break
    print(i)
    i += 1
```

- a) 5 6 7 8 9 10
- b) 5 6 7 8**
- c) 5 6
- d) error

Explanation: 0O11 is an octal number.

59. What is the output of the following?

```
i = 5
```

```
while True:
```

```
    if i%009 == 0:
```

```
        break
```

```
    print(i)
```

```
    i += 1
```

a) 5 6 7 8

b) 5 6 7 8 9

c) 5 6 7 8 9 10 11 12 13 14 15

d) error

Explanation: 9 isn't allowed in an octal number.

60. What is the output of the following?

```
i = 1
```

```
while True:
```

```
    if i%2 == 0:
```

```
        break
```

```
    print(i)
```

```
    i += 2
```

a) 1

b) 1 2

c) 1 2 3 4 5 6 ...

d) 1 3 5 7 9 11 ...

Explanation: The loop does not terminate since i is never an even number.

61. What is the output of the following?

```
i = 2
```

```
while True:
```

```
    if i%3 == 0:
```

```
        break
```

```
    print(i)
```

```
    i += 2
```

a) 2 4 6 8 10 ...

b) 2 4

c) 2 3

d) error

Explanation: The numbers 2 and 4 are printed. The next value of i is 6 which is divisible by 3 and hence control exits the loop.

62. What is the output of the following?

```
i = 1
```

```
while False:
```

```
    if i%2 == 0:
```

```
        break
```

```
    print(i)
```

```
    i += 2
```

a) 1

b) 1 3 5 7 ...

c) 1 2 3 4 ...

d) none of the mentioned

Explanation: Control does not enter the loop because of False.

63. What is the output of the following?

```
True = False
```

```
while True:
```

```
    print(True)
```

```
    break
```

a) True

b) False

c) None

d) none of the mentioned

Explanation: SyntaxError, True is a keyword and it's value cannot be changed

64. What is the output of the following?

```
i = 0
```

```
while i < 5:
```

```
    print(i)
```

```
    i += 1
```

```
    if i == 3:
```

```
        break
```

```
else:
```

```
    print(0)
```

a) 0 1 2 0

b) 0 1 2

c) error

d) none of the mentioned

Explanation: The else part is not executed if control breaks out of the loop.

65. What is the output of the following?

```
i = 0
```

```
while i < 3:
```

```
    print(i)
```

```
    i += 1
```

```
else:
```

```
    print(0)
```

a) 0 1 2 3 0

b) 0 1 2 0

c) 0 1 2

c) error

Explanation: The else part is executed when the condition in the while statement is false.

66. What is the output of the following?

```
x = "abcdef"
```

```
while i in x:
```

```
    print(i, end=" ")
```

a) a b c d e f

b) abcdef

c) i i i i i ...

d) error

Explanation: NameError, i is not defined.

67. What is the output of the following?

```
x = "abcdef"
```

```
i = "i"
```

```
while i in x:
```

```
    print(i, end=" ")
```

a) no output

b) i i i i i ...

c) a b c d e f

d) abcdef

Explanation: "i" is not in "abcdef".

68. What is the output of the following?

```
x = "abcdef"
```

```
i = "a"
```

```
while i in x:
```

```
    print(i, end = " ")
```

a) no output

b) i i i i i ...

c) a a a a a ...

d) a b c d e f

Explanation: As the value of i or x isn't changing, the condition will always evaluate to True.

69. What is the output of the following?

```
x = "abcdef"
```

```
i = "a"
```

```
while i in x:
```

```
    print('i', end = " ")
```

a) no output

b) i i i i i ...

c) a a a a a ...

d) a b c d e f

Explanation: As the value of i or x isn't changing, the condition will always evaluate to True.

70. What is the output of the following?

```
x = "abcdef"
```

```
i = "a"
```

```
while i in x:
```

```
    x = x[:-1]
```

```
    print(i, end = " ")
```

a) i i i i i

b) a a a a a

c) a a a a a

d) none of the mentioned

Explanation: The string x is being shortened by one character in each iteration.

71. What is the output of the following?

```
x = "abcdef"
```

```
i = "a"
```

```
while i in x[:-1]:
```

```
    print(i, end = " ")
```

a) a a a a a

b) a a a a a

c) a a a a a ...

d) a

Explanation: String x is not being altered and i is in x[:-1].

72. What is the output of the following?

```
x = "abcdef"
i = "a"
while i in x:
    x = x[1:]
    print(i, end = " ")
```

a) a a a a a a

b) a

c) no output

d) error

Explanation: The string x is being shortened by one character in each iteration.

73. What is the output of the following?

```
x = "abcdef"
i = "a"
while i in x[1:]:
    print(i, end = " ")
```

a) a a a a a a

b) a

c) no output

d) error

Explanation: i is not in x[1:].

74. What is the output of the following?

```
x = 'abcd'
for i in x:
    print(i)
    x.upper()
```

a) a B C D

b) a b c d

c) A B C D

d) error

Explanation: Changes do not happen in-place, rather a new instance of the string is returned.

75. What is the output of the following?

```
x = 'abcd'
for i in x:
    print(i.upper())
```

a) a b c d

b) A B C D

c) a B C D

d) error

Explanation: The instance of the string returned by upper() is being printed.

76. What is the output of the following?

```
x = 'abcd'
for i in range(x):
    print(i)
```

a) a b c d

b) 0 1 2 3

c) error

d) none of the mentioned

Explanation: range(str) is not allowed.

77. What is the output of the following?

```
x = 'abcd'
for i in range(len(x)):
    print(i)
```

- a) a b c d
- b) 0 1 2 3**
- c) error
- d) 1 2 3 4

Explanation: i takes values 0, 1, 2 and 3.

78. What is the output of the following?

```
x = 'abcd'
for i in range(len(x)):
    print(i.upper())
```

- a) a b c d
- b) 0 1 2 3
- c) error**
- d) 1 2 3 4

Explanation: Objects of type int have no attribute upper().

79. What is the output of the following?

```
x = 'abcd'
for i in range(len(x)):
    i.upper()
print (x)
```

- a) a b c d
- b) 0 1 2 3
- c) error**
- d) none of the mentioned

Explanation: Objects of type int have no attribute upper().

80. What is the output of the following?

```
x = 'abcd'
for i in range(len(x)):
    x[i].upper()
print (x)
```

- a) abcd**
- b) ABCD
- c) error
- d) none of the mentioned

Explanation: Changes do not happen in-place, rather a new instance of the string is returned.

81. What is the output of the following?

```
x = 'abcd'
for i in range(len(x)):
    i[x].upper()
print (x)
```

- a) abcd
- b) ABCD
- c) error**
- d) none of the mentioned

Explanation: Objects of type int aren't subscriptable. However, if the statement was x[i], an error would not have been thrown.

82. What is the output of the following?

```
x = 'abcd'
for i in range(len(x)):
    x = 'a'
    print(x)
```

- a) a
- b) abcd abcd abcd
- c) a a a a
- d) none of the mentioned

Explanation: range() is computed only at the time of entering the loop.

83. What is the output of the following?

```
x = 'abcd'
for i in range(len(x)):
    print(x)
    x = 'a'
```

- a) a
- b) abcd abcd abcd abcd
- c) a a a a
- d) none of the mentioned

Explanation: abcd a a a a is the output as x is modified only after 'abcd' has been printed once

84. What is the output of the following?

```
x = 123
for i in x:
    print(i)
```

- a) 1 2 3
- b) 123
- c) error
- d) none of the mentioned

Explanation: Objects of type int are not iterable.

85. What is the output of the following?

```
d = {0: 'a', 1: 'b', 2: 'c'}
for i in d:
    print(i)
```

- a) 0 1 2
- b) a b c
- c) 0 a 1 b 2 c
- d) none of the mentioned

Explanation: Loops over the keys of the dictionary.

86. What is the output of the following?

```
d = {0: 'a', 1: 'b', 2: 'c'}
for x, y in d:
    print(x, y)
```

- a) 0 1 2
- b) a b c
- c) 0 a 1 b 2 c
- d) none of the mentioned

Explanation: Error, objects of type int aren't iterable.

87. What is the output of the following?

```
d = {0: 'a', 1: 'b', 2: 'c'}  
for x, y in d.items():  
    print(x, y)
```

- a) 0 1 2
- b) a b c
- c) 0 a 1 b 2 c**
- d) none of the mentioned

Explanation: Loops over key, value pairs.

88. What is the output of the following?

```
d = {0: 'a', 1: 'b', 2: 'c'}  
for x in d.keys():  
    print(d[x])
```

- a) 0 1 2
- b) a b c**
- c) 0 a 1 b 2 c
- d) none of the mentioned

Explanation: Loops over the keys and prints the values.

89. What is the output of the following?

```
d = {0: 'a', 1: 'b', 2: 'c'}  
for x in d.values():  
    print(x)
```

- a) 0 1 2
- b) a b c**
- c) 0 a 1 b 2 c
- d) none of the mentioned

Explanation: Loops over the values.

90. What is the output of the following?

```
d = {0: 'a', 1: 'b', 2: 'c'}  
for x in d.values():  
    print(d[x])
```

- a) 0 1 2
- b) a b c
- c) 0 a 1 b 2 c
- d) none of the mentioned**

Explanation: Causes a KeyError.

91. What is the output of the following?

```
d = {0, 1, 2}  
for x in d.values():  
    print(x)
```

- a) 0 1 2
- b) None None None
- c) error**
- d) none of the mentioned

Explanation: Objects of type set have no attribute values.

92. What is the output of the following?

```
d = {0, 1, 2}
for x in d:
    print(x)
```

- a) 0 1 2
- b) {0, 1, 2} {0, 1, 2} {0, 1, 2}
- c) error
- d) none of the mentioned

Explanation: Loops over the elements of the set and prints them.

93. What is the output of the following?

```
d = {0, 1, 2}
for x in d:
    print(d.add(x))
```

- a) 0 1 2
- b) 0 1 2 0 1 2 0 1 2 ...
- c) None None None
- d) none of the mentioned

Explanation: Variable x takes the values 0, 1 and 2. set.add() returns None which is printed.

94. What is the output of the following?

```
for i in range(0):
    print(i)
```

- a) 0
- b) no output
- c) error
- d) none of the mentioned

Explanation: range(0) is empty

95. What is the output of the following?

```
for i in range(2.0):
    print(i)
```

- a) 0.0 1.0
- b) 0 1
- c) error
- d) none of the mentioned

Explanation: Object of type float cannot be interpreted as an integer.

96. What is the output of the following?

```
for i in range(int(2.0)):
    print(i)
```

- a) 0.0 1.0
- b) 0 1
- c) error
- d) none of the mentioned

Explanation: range(int(2.0)) is the same as range(2).

97. What is the output of the following?

```
for i in range(float('inf')):
    print(i)
```

- a) 0.0 0.1 0.2 0.3 ...
- b) 0 1 2 3 ...
- c) 0.0 1.0 2.0 3.0 ...
- d) none of the mentioned

Explanation: Error, objects of type float cannot be interpreted as an integer.

98. What is the output of the following?

```
for i in range(int(float('inf'))):  
    print (i)
```

- a) 0.0 0.1 0.2 0.3 ...
- b) 0 1 2 3 ...
- c) 0.0 1.0 2.0 3.0 ...

d) none of the mentioned

Explanation: OverflowError, cannot convert float infinity to integer.

99. What is the output of the following?

```
for i in [1, 2, 3, 4][::-1]:  
    print (i)
```

- a) 1 2 3 4
- b) 4 3 2 1
- c) error
- d) none of the mentioned

Explanation: [::-1] reverses the list.

100. What is the output of the following?

```
for i in ''.join(reversed(list('abcd'))):  
    print (i)
```

- a) a b c d
- b) d c b a
- c) error
- d) none of the mentioned

Explanation: ‘ ‘.join(reversed(list(‘abcd’))) reverses a string.

101. What is the output of the following?

```
for i in 'abcd'[::-1]:  
    print (i)
```

- a) a b c d
- b) d c b a
- c) error
- d) none of the mentioned

Explanation: [::-1] reverses the string.

102. What is the output of the following?

```
for i in "":  
    print (i)
```

- a) None
- b) (nothing is printed)
- c) error
- d) none of the mentioned

Explanation: The string does not have any character to loop over.

103. What is the output of the following?

```
x = 2  
for i in range(x):  
    x += 1  
    print (x)
```

- a) 0 1 2 3 4 ...
- b) 0 1
- c) 3 4
- d) 0 1 2 3

Explanation: Variable x is incremented and printed twice.

104. What is the output of the following?

```
x = 2
for i in range(x):
    x -= 2
    print (x)
```

a) 0 1 2 3 4 ...

b) 0 -2

c) 0

d) error

Explanation: The loop is entered twice

105. What is the output of the following?

```
for i in range(10):
    if i == 5:
        break
    else:
        print(i)
else:
    print("Here")
```

a) 0 1 2 3 4 Here

b) 0 1 2 3 4 5 Here

c) 0 1 2 3 4

d) 1 2 3 4 5

Explanation: The else part is executed if control doesn't break out of the loop.

106. What is the output of the following?

```
for i in range(5):
    if i == 5:
        break
    else:
        print(i)
else:
    print("Here")
```

a) 0 1 2 3 4 Here

b) 0 1 2 3 4 5 Here

c) 0 1 2 3 4

d) 1 2 3 4 5

Explanation: The else part is executed if control doesn't break out of the loop.

107. What is the output of the following?

```
x = (i for i in range(3))
for i in x:
    print(i)
```

a) 0 1 2

b) error

c) 0 1 2 0 1 2

d) none of the mentioned

Explanation: The first statement creates a generator object.

108. What is the output of the following?

```
x = (i for i in range(3))
for i in x:
    print(i)
for i in x:
    print(i)
```

a) 0 1 2

b) error

c) 0 1 2 0 1 2

d) none of the mentioned

Explanation: We can loop over a generator object only once.

109. What is the output of the following?

```
string = "my name is x"
for i in string:
    print (i, end=" ", )
```

a) m, y, , n, a, m, e, , i, s, , x,

b) m, y, , n, a, m, e, , i, s, , x

c) my, name, is, x,

d) error

Explanation: Variable i takes the value of one character at a time.

110. What is the output of the following?

```
string = "my name is x"
for i in string.split():
    print (i, end=" ", )
```

a) m, y, , n, a, m, e, , i, s, , x,

b) m, y, , n, a, m, e, , i, s, , x

c) my, name, is, x,

d) error

Explanation: Variable i takes the value of one word at a time.

111. What is the output of the following?

```
a = [0, 1, 2, 3]
for a[-1] in a:
    print(a[-1])
```

a) 0 1 2 3

b) 0 1 2 2

c) 3 3 3 3

d) error

Explanation: The value of a[-1] changes in each iteration.

112. What is the output of the following?

```
a = [0, 1, 2, 3]
for a[0] in a:
    print(a[0])
```

a) 0 1 2 3

b) 0 1 2 2

c) 3 3 3 3

d) error

Explanation: The value of a[0] changes in each iteration. Since the first value that it takes is itself, there is no visible error in the current example.